

MBT App Application Architecture Document

focusing on API and other integrations

Date: 08 April 2026

Classification: Internal / Confidential

Version: 1.0

Table of Contents

1. Executive Summary
2. System Overview
3. Architecture Diagram
4. Component Details
 - 4.1 Backend API (Laravel)
 - 4.2 Admin Panel (Laravel)
 - 4.3 iOS Mobile Application
 - 4.4 Android Mobile Application
5. External Integrations
 - 5.1 SAP File Integration
 - 5.2 Payslip SOAP API (SAP BTP)
 - 5.3 SMS Gateway (Posta Güvercini)
 - 5.4 Azure AD / MSAL Authentication
 - 5.5 Telegram Bot Logging
6. API Endpoints Reference
7. Data Flow Diagrams
8. Security Considerations
9. Infrastructure & Deployment

1. Executive Summary

MBT App is an enterprise mobile application developed for Mercedes-Benz Türk A.Ş. employees. The application provides a range of HR and corporate services including employee authentication, payslip viewing, work hour tracking, work calendar management, meal menus, shuttle app forwarding, corporate news, social clubs, digital business cards, and internal phone directories.

The system is composed of four main components: a Laravel-based REST API backend, a Laravel-based Admin Panel for content management, and native mobile applications for iOS (Swift/Objective-C) and Android (Java/Kotlin). These components integrate with external systems including SAP (via file exchange and SOAP APIs), Microsoft Azure AD (for MSAL-based authentication) and Posta Güvercini (SMS gateway for OTP).

2. System Overview

Component	Technology Stack	Host / IP	Purpose
Backend API	PHP 7.x / Laravel	Staging: https://biziapp-test.app.daimlertruck.com/bizizBackend 95.214.97.107 Prod: https://bizizapp.com/bizizBackend 95.214.97.108	REST API serving mobile apps
Admin Panel	PHP / Laravel, Livewire, Blade	Staging: https://biziapp-test.app.daimlertruck.com/bizizPanel 95.214.97.107 Prod: https://bizizapp.com/bizizPanel 95.214.97.108	Content & configuration management
MySQL Database	MySQL (utf8mb4_unicode_ci)	Staging : 192.168.24.15:3306 Prod : 192.168.24.25:3306	Primary data store (shared by Backend & Panel)

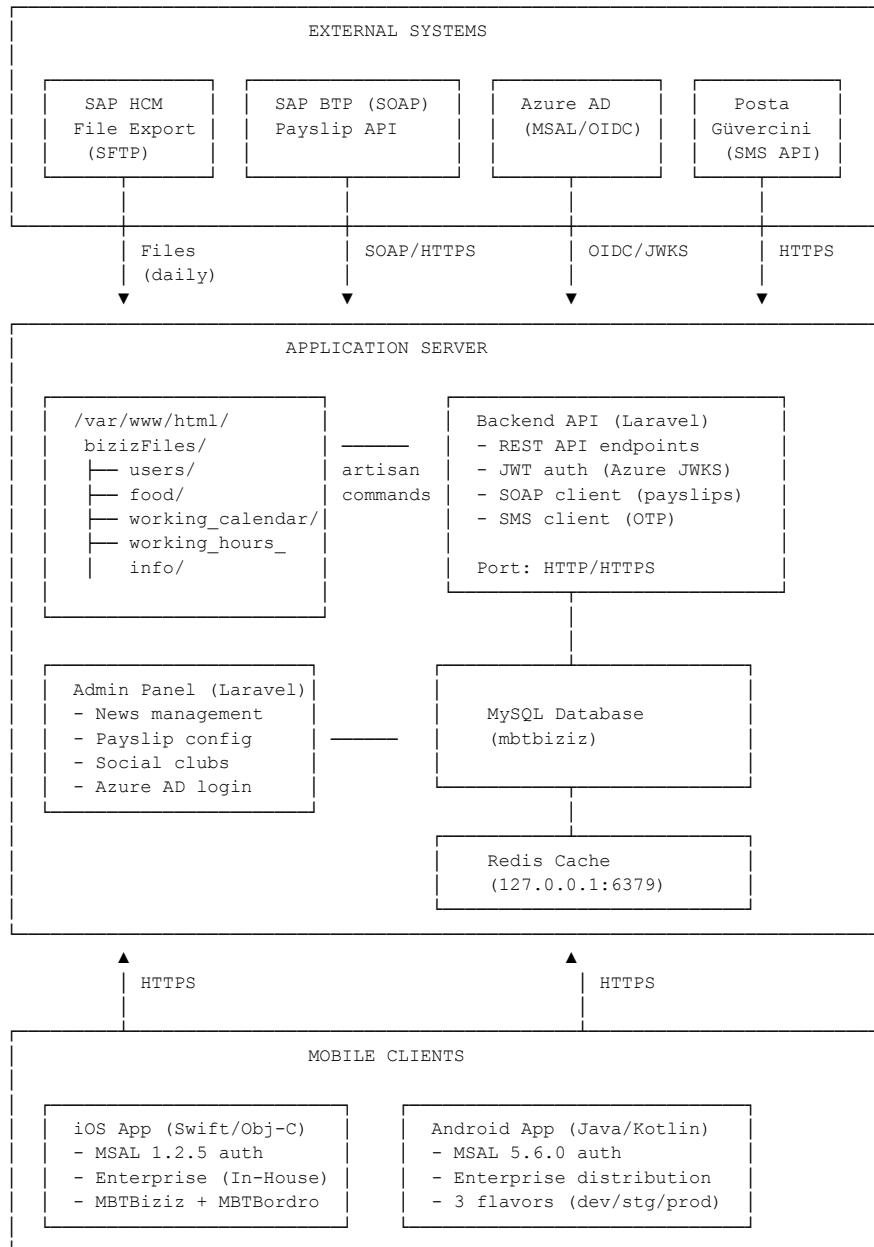
		(No public internet access)	
Redis Cache	Redis (Predis client)	127.0.0.1:6379 (localhost of server machine)	Session & data caching (food menu, etc.)
iOS App	Swift 4 / Obj-C, CocoaPods, MSAL 1.2.5	N/A (client)	Native iOS mobile application
Android App	Java/Kotlin, Gradle, MSAL 5.6.0	N/A (client)	Native Android mobile application

Key Technology Details

Item	Detail
Backend Framework	Laravel (Lumen-based lightweight API)
Database Charset	utf8mb4_unicode_ci
Authentication	JWT tokens verified via Azure AD JWKS public keys
iOS Bundle IDs	Production: com.daimlertruck.dtag.internal.ios.mbt.test Staging: com.daimlertruck.dtag.internal.ios.mbt.test2
Android Package	com.daimlertruck.dtag.internal.android.mbt.test
iOS Signing	Enterprise (In-House) distribution, Team ID: J7U9JQP7Q6
iOS Targets	MBTBiziz-Production, MBTBiziz-Staging, MBTBordro-Production, MBTBordro-Staging
Android Flavors	prod, staging, dev

3. Architecture Diagram

Below is a textual representation of the system architecture.



4. Component Details

4.1 Backend API (Laravel)

The backend API is built on Laravel (Lumen variant) and serves as the central REST API for both iOS and Android mobile applications. It manages user authentication, data aggregation from SAP file imports, payslip retrieval via SOAP calls, and push notification orchestration.

Scheduled Tasks (Cron)

Command	Schedule	File Source	Target
set:users	Daily 07:05 (Europe/Istanbul)	/var/www/html/biziz Files/users/mbt_pers data_{ddmmYYYY}.tx t	DB: users table
set:working_hours_info	Daily 07:08	/var/www/html/biziz Files/working_hours_ info/mbt_my_timeda ta_{ddmmYYYY}.txt	DB: working_hours_info
set:working_calendar	Daily 07:08	/var/www/html/biziz Files/working_calend ar/mbt_dvm_dvms_{ ddmmYYYY}.txt	DB: working_calendar
set:food_menu	Daily 07:09 + Hourly	/var/www/html/biziz Files/food/Yemek_*. xlsx	Redis cache
set:food_shuttle	Daily 07:09 + Hourly	/var/www/html/biziz Files/food/Yemek_Se rvis.xlsx	Redis cache

Authentication Flow

The backend verifies JWT tokens issued by Azure AD. On each authenticated request, the token is validated against Azure AD's JWKS endpoint:

- JWKS URL: <https://login.microsoftonline.com/505cca53-5750-4134-9501-8d52d5df3cd1/discovery/v2.0/keys>
- Tenant ID: 505cca53-5750-4134-9501-8d52d5df3cd1

4.2 Admin Panel (Laravel)

The Admin Panel is a separate Laravel web application used by authorized administrators to manage content and configurations for the mobile app. Authentication is via Azure AD OIDC (OpenID Connect) using the rootinc/laravel-azure-middleware package.

Panel Capabilities

- News/Announcement management with push notification dispatch
- Payslip feature toggle (activate/deactivate) and period messaging
- Social clubs directory management

- Internal phone directory management
- Media (image/video) content management
- User feedback review and email routing configuration
- Digital business card management
- Discount code management
- About Us / App description editing

Panel Azure AD Configuration

Parameter	Value
Tenant ID	505cca53-5750-4134-9501-8d52d5df3cd1
Client ID	Configured via AZURE_CLIENT_ID env variable
Client Secret	Configured via AZURE_CLIENT_SECRET env variable
Resource	https://graph.microsoft.com/
Scope	openid
Auth Package	rootinc/laravel-azure-middleware ^0.9.10

4.3 iOS Mobile Application

The iOS application is a native app built with Swift 4 and Objective-C using the VIP (View-Interactor-Presenter) architecture pattern. It is distributed via Apple Enterprise (In-House) program under team ID J7U9JQP7Q6 (MERCEDES BENZ TURK ANONIM SIRKETI).

iOS Build Targets

Target	Bundle ID	Purpose
MBTBordro-Production	com.daimlertruck.dtag.internal.ios.mbt.test	Payslip app - Production
MBTBordro-Staging	com.daimlertruck.dtag.internal.ios.mbt.test2	Payslip app - Staging

iOS Dependencies (CocoaPods)

Library	Purpose
MSAL 1.2.5	Microsoft Authentication Library for Azure AD
Alamofire	HTTP networking
AlamofireImage	Image loading/caching with Alamofire
Moya	Network abstraction layer on top of Alamofire
Kingfisher	Image downloading and caching
Marshal	JSON marshalling/unmarshalling
ImageSlideshow	Image slideshow UI component
PureLayout	Auto Layout helper
AnyFormatKit	Text field formatting (phone numbers)
EZSwiftExtensions	Swift utility extensions

iOS MSAL Configuration

Environment	Client ID	Redirect URI	Scopes
Production	41b14095-c137-4f5d-b300-accd37cbb4aa	msauth.com.daimlert ruck.dtag.internal.ios .mbt.test://auth	api://48252d22- 0987-4d84-b1d9- 00468ec9d424/Read
Staging/Test	e97a3881-d802- 433b-8529- 96e0e801e346	msauth.com.daimlert ruck.dtag.internal.ios .mbt.test2://auth	api://910155c2-0cc9- 4d21-a48b- 4c49c99f8128/Read

4.4 Android Mobile Application

The Android application is a native app built with Java/Kotlin using Gradle. It supports three product flavors (dev, staging, prod) and uses MSAL 5.6.0 for Azure AD authentication.

Android Build Configuration

Flavor	Application ID	Purpose
prod	com.daimlertruck.dtag.intern al.android.mbt.test	Production
staging	com.daimlertruck.dtag.intern al.android.mbt.test (staging suffix)	Staging

Android MSAL Configuration

Parameter	Value
Client ID	bc50644f-7099-4737-bcd1-a2791ba4930b
Tenant ID	505cca53-5750-4134-9501-8d52d5df3cd1
Redirect URI	msauth://com.daimlertruck.dtag.internal.and roid.mbt.test/...
Account Mode	SINGLE
Authority Type	AAD (Azure Active Directory)
MSAL SDK Version	5.6.0

5. External Integrations

5.1 SAP File Integration

SAP HCM exports employee and operational data as flat files (tab-separated text and Excel) which are deposited via SFTP to the application server. Laravel artisan scheduled commands parse these files and import the data into the MySQL database or Redis cache.

File Transfer Details

Parameter	Value
Transfer Method	SFTP (files deposited by SAP to local server)
Base Directory	/var/www/html/bizizFiles/

Frequency	Daily (files generated with date stamp in filename)
File Encoding	UTF-8 (with automatic encoding detection and conversion)

SAP File Paths & Data Mapping

Data Type	Path	File Pattern	Format	Fields
User Info	/var/www/html/bizizFiles/users/	mbt_persdata_{ddmmYYYY}.txt	Tab-separated, 16 columns	Register No, Username, Name, Cost Center, Birthday, Type (Blue/White Collar), Location, Phone, Email, Title, Company Code
Working Hours	/var/www/html/bizizFiles/working_hours_info/	mbt_my_timedata_{ddmmYYYY}.txt	Tab-separated, 6 columns	Register No, Date, Hours Info, Plus Hours (NÇ), Minus Hours (DVS), Total Hours (FM+)
Work Calendar	/var/www/html/bizizFiles/working_calendar/	mbt_dvm_dvms_{ddmmYYYY}.txt	Tab-separated, 4 columns	Register No, Date, Attendance Type Code, Attendance Type Name
Meal Menu	/var/www/html/bizizFiles/food/	Yemek_*.xlsx	Excel (XLSX), 7 columns	Date, Food Name, Calories, Protein, Fat, Carbohydrate, Notes

5.2 Payslip SOAP API (SAP BTP)

Payslip (Bordro) data is retrieved in real-time from SAP via a SOAP web service call. The integration goes through SAP BTP (Business Technology Platform) with OAuth2 client credentials authentication followed by the SOAP call.

Payslip API Details

Parameter	Value
Protocol	SOAP over HTTPS
WSDL Operation	osPayslipBase64Get

RFC Function	ZCOFFIX_FM_PAYSLIP_BASE64_GET
SOAP Action	http://sap.com/xi/WebService/soap1.1
SAP BTP SOAP Endpoint (Production)	https://central-cpi-prod-aws-jiswscnu.it-cpi018-rt.cfapps.eu10-003.hana.ondemand.com/cxf/Cloudoffix_to_MBT_PayslipBase64Get
SAP BTP OAuth2 Token URL	https://central-cpi-prod-aws-jiswscnu.authentication.eu10.hana.ondemand.com/oauth/token
SAP BTP OAuth2 Client ID	sb-c2d1f0cb-5ca2-4f7b-91aa-e655209f3680!b184378 it-rt-central-cpi-prod-aws-jiswscnu!b117912
SAP BTP OAuth2 Client Secret	(stored in .env as BORDRO_BTP_CLIENT_SECRET)
SAP SOAP User (RFC)	RFCBIZIZ
On-Premise SAP PI Endpoint (Legacy)	https://a152pmdjscs.tr152.corpintra.net:443/XISOAPAdapter/MessageServlet
Dev/Test Endpoint	https://dtagdevapi.test.apimanagement.eu10.hana.ondemand.com/MBT/XISOAPAdapter/MessageServlet
HTTP Client	GuzzleHttp\Client

Payslip SOAP Request Parameters

Parameter	Format	Description
IV_PERIOD	6 digits (YYYYMM)	Payslip period (e.g., 202604)
IV_PERNR	8 digits	Employee personnel number (SAP register number)

Payslip SOAP Response

Field	Description
EV_BASE64	Payslip PDF document encoded in Base64
EV_MESSAGE	Response message (success or error description)
EV_TYPE	Response type indicator

Payslip Authentication Flow

1. User requests payslip via mobile app → Backend /api/v1/payslip/request-otp
2. Backend sends OTP via SMS (Posta Güvercini) to user's registered phone
3. User submits OTP → Backend /api/v1/payslip/verify-otp (validates PIN)
4. User requests specific payslip → Backend /api/v1/payslip/fetch
5. Backend obtains OAuth2 token from SAP BTP token URL (client_credentials grant)
6. Backend makes SOAP call to BTP endpoint with Bearer token + SOAP XML

7. SAP returns Base64-encoded PDF payslip
8. Backend returns PDF data to mobile app for display

5.3 SMS Gateway — Posta Güvercini

SMS messages are sent for OTP (One-Time Password) verification during login and payslip access. Two separate Posta Güvercini endpoints are used.

SMS Service Endpoints

Purpose	URL	Method	Auth	Message Template
Login OTP	https://www.postaguvercini.com/api_http/sendsms.asp	HTTP GET	User/Password in query params	MBT App pin kodunuz: {pinCode}
Payslip OTP	https://otpsms.postaguvercini.com/api_http/sendsms.asp	HTTP GET	User/Password via env (PG_USER, PG_PASSWORD)	MBT App bordro görüntüleme doğrulama kodunuz: {otp}

SMS API Parameters

Parameter	Description
user	Posta Güvercini account username (PG_USER: Mercedes1357)
password	Posta Güvercini account password (PG_PASSWORD in .env)
gsm	Recipient phone number (10 digits with leading 0)
text	SMS message body

Success response format: `errno=0&errtext=&message_id=XXXXX&charge=1`

5.4 Azure AD / MSAL Authentication

All user authentication across the platform uses Microsoft Azure Active Directory via MSAL (Microsoft Authentication Library). The Azure AD tenant is shared across all components.

Azure AD Configuration Summary

Parameter	Value
Tenant ID	505cca53-5750-4134-9501-8d52d5df3cd1
Organization	MERCEDES BENZ TURK ANONIM SIRKETI
Auth Endpoint	https://login.microsoftonline.com/505cca53-5750-4134-9501-8d52d5df3cd1
JWKS Endpoint	https://login.microsoftonline.com/505cca53-5750-4134-9501-8d52d5df3cd1/discovery/v2.0/keys

Client Application Registrations

Component	Client ID	Scopes / Resource
iOS Production	41b14095-c137-4f5d-b300-accd37cbb4aa	api://48252d22-0987-4d84-b1d9-00468ec9d424/Read
iOS Staging	e97a3881-d802-433b-8529-96e0e801e346	api://910155c2-0cc9-4d21-a48b-4c49c99f8128/Read
Android	bc50644f-7099-4737-bcd1-a2791ba4930b	AAD authority with tenant
Admin Panel	Via AZURE_CLIENT_ID env variable	https://graph.microsoft.com/ (openid scope)

Authentication Flow

1. Mobile app initiates MSAL interactive login → Azure AD login page displayed
2. User authenticates with corporate credentials → Azure AD issues JWT access token
3. Mobile app sends JWT token with each API request (Authorization: Bearer header)
4. Backend fetches Azure AD JWKS public keys and validates the JWT signature
5. If valid, backend extracts user identity and processes the request
6. Token refresh is handled by MSAL SDK automatically on both platforms

6. API Endpoints Reference

6.1 Public Endpoints (No Authentication)

Method	URI	Controller	Description
POST	/api/v1/init	InitController@init	App initialization, version check
POST	/api/v1/login	LoginController@login	Verify OTP and authenticate user
POST	/api/v1/appStartup	LoginController@appStartup	App startup flow
GET	/dc/{uuid}	DigitalCard	View digital business card
GET	/dc/{uuid}/downloadVcf	DigitalCard	Download vCard file

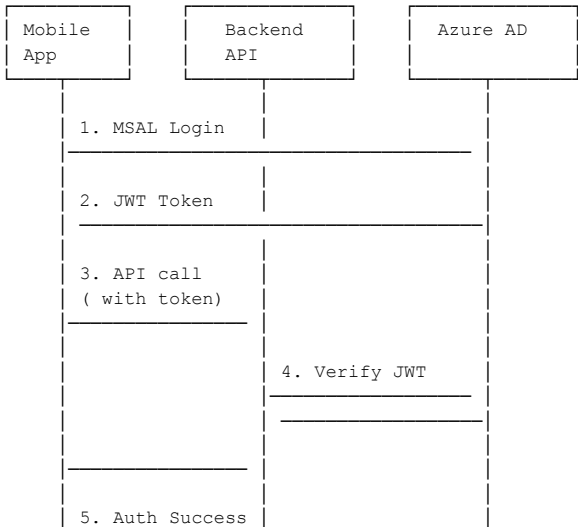
6.2 Protected Endpoints (JWT Authentication Required)

Method	URI	Description
POST	/api/v1/newsList	Get paginated news list
GET	/api/v1/newsDetail/{id}	Get news article detail
GET	/api/v1/birthdayList	Get employee birthday list
POST	/api/v1/saveDeviceInfo	Register device for push notifications

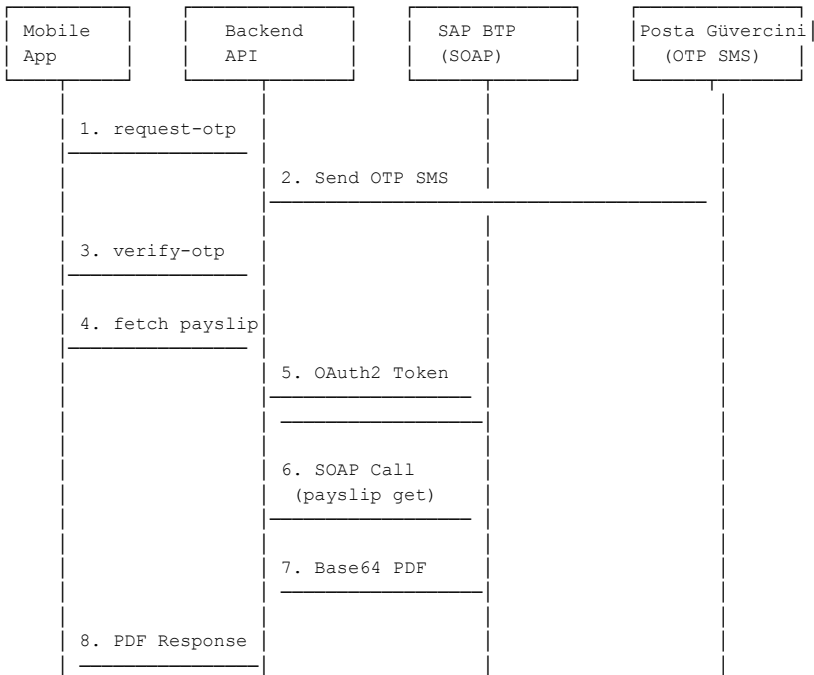
POST	/api/v1/deleteDeviceInfo	Unregister device for push notifications
POST	/api/v1/notificationList	Get notification list
POST	/api/v1/deleteNotification	Delete a notification
GET	/api/v1/notificationBadgeCount	Get unread notification count
GET	/api/v1/foodMenu	Get weekly meal menu
GET	/api/v1/profile	Get employee profile
GET	/api/v1/workCalendar/{date}	Get work calendar for date
GET	/api/v1/yearlyWorkHours/{year}	Get yearly work hours summary
GET	/api/v1/monthlyWorkHours/{year}/{month}	Get monthly work hours detail
GET	/api/v1/notificationSettings	Get notification preferences
POST	/api/v1/changeNotificationSetting	Update notification preference
GET	/api/v1/maps	Get campus map data
GET	/api/v1/signOut	Sign out user
GET	/api/v1/userConfig	Get user-specific configuration
GET	/api/v1/getDiscountCode/{newsId}	Get discount code for news item
GET	/api/v1/socialClubLocs	Get social club locations
GET	/api/v1/socialClubs/{loc_id}	Get social clubs at location
GET	/api/v1/phoneLocs	Get phone directory locations
GET	/api/v1/phones/{loc_id}	Get phone list at location
GET	/api/v1/medias	Get media content
POST	/api/v1/submitFeedback	Submit user feedback
GET	/api/v1/payslip/isActive	Check if payslip feature is active
POST	/api/v1/payslip/request-otp	Request OTP for payslip access
POST	/api/v1/payslip/verify-otp	Verify payslip access OTP
POST	/api/v1/payslip/fetch	Fetch payslip PDF (via SAP SOAP)
GET	/api/v1/getUserBusinessCardState	Get digital card state
GET	/api/v1/activateDigitalCard	Activate digital card
GET	/api/v1/deactivateDigitalCard	Deactivate digital card
POST	/api/v1/menuIncrement	Track menu view analytics

7. Data Flow Diagrams

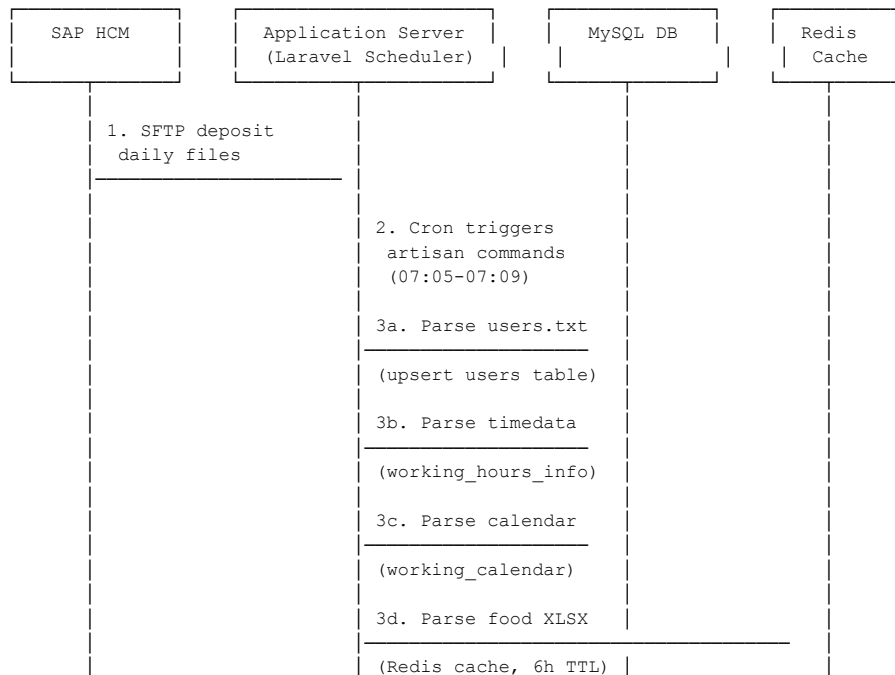
7.1 User Login Flow



7.2 Payslip Access Flow



7.3 SAP File Integration Flow



8. Security Considerations

8.1 Authentication & Authorization

- All API endpoints (except init, login, checkPhone) require JWT Bearer token authentication
- JWT tokens are validated against Azure AD JWKS public keys (RS256 signature verification)
- Phone-based OTP verification adds a second factor for payslip access
- Admin Panel access requires Azure AD OIDC authentication

8.2 Data Protection

- All external API calls use HTTPS/TLS encryption
- Payslip SOAP calls are authenticated with OAuth2 client credentials + Basic auth
- Sensitive configuration (DB credentials, API keys, Azure secrets) stored in environment variables
- SMS OTP codes provide time-limited access for payslip viewing (5 minutes)
- CSP (Content Security Policy) headers configured via csp.php config
- Security headers middleware applied to all API routes

8.3 Infrastructure Security

- SAP file transfers via SFTP (SSH-based encrypted file transfer)
- MySQL database server (192.168.24.25) on private network — no public internet access
- Redis cache on localhost only (127.0.0.1:6379) — not exposed to network

- Enterprise (In-House) app distribution for iOS (no App Store exposure)
- Android signing with dedicated keystore
- SMTP email via smtp.gmail.com:587 with TLS encryption
- App prod download URL: https://bizizapp.com/mbt_app.html (public-facing)
 - IOS direct link for staging:
https://biziapp-test.app.daimlertruck.com/app/bordro/MBT_App.ipa
 - Android direct link for staging:
https://biziapp-test.app.daimlertruck.com/app/bordro/mbt_app.apk
- Digital card service
 - prod: URL: <https://bizizapp.com/dk> (public-facing)
 - staging: URL: <https://biziapp-test.app.daimlertruck.com/dk> (public-facing)

9. Infrastructure & Deployment

9.1 Server Environment

Component	IP / Host	Port	Access	Notes
Staging App Server	https://biziapp-test.app.daimlertruck.com 95.214.97.107	22 (SFTP), 80/443 (HTTP/S)	Internal network only	Backend API + Admin Panel + SAP file storage
Production App Server	https://bizizapp.com 95.214.97.108	22 (SFTP), 80/443 (HTTP/S)	Internal network only	Backend API + Admin Panel + SAP file storage
MySQL Database Server	192.168.24.25	3306	Internal network only, NO public internet	Shared by Backend API and Admin Panel
Redis Cache	127.0.0.1 (localhost)	6379	Localhost only	On each app server, not network-exposed
SAP File Storage	Local filesystem	N/A	Local filesystem	/var/www/html/bizizFiles/ (SAP deposits via SFTP)
App Distribution (IPA/APK)	staging and prod servers	HTTP/S	Internal network	/var/www/html/app/bordro/
Public Web (bizizapp.com)	Prod: bizizapp.com Staging : https://biziapp-test.app.daimlertruck.com	443	Public internet	App download page + Digital Card service + admin dashboard

9.2 External Service Dependencies (Outbound from App Servers)

Service	Endpoint / Host	Port	Protocol	Direction	Purpose
Azure AD	login.microsoftonline.com	443	HTTPS (OIDC/JWKS)	Outbound	JWT token verification (JWKS public keys)
SAP BTP Payslip (SOAP)	central-cpi-prod-aws-jiswscnu.it-cpi018-rt.cfapps.eu10-003.hana.ondemand.com	443	SOAP over HTTPS	Outbound	Payslip PDF retrieval
SAP BTP OAuth2	central-cpi-prod-aws-jiswscnu.authentication.eu10.hana.ondemand.com	443	HTTPS POST	Outbound	OAuth2 token for payslip API
SAP PI (Legacy)	a152pmdjcs.tr152.corpint.ra.net	443	SOAP over HTTPS	Outbound	Legacy payslip endpoint
Posta Güvercini (Login SMS)	www.postaguvercini.com	443	HTTPS GET	Outbound	Login OTP SMS sending
Posta Güvercini (Payslip SMS)	otpsms.postaguvercini.com	443	HTTPS GET	Outbound	Payslip OTP SMS sending
Gmail SMTP	smtp.gmail.com	587	SMTP/TLS	Outbound	Email notifications

9.3 Network Topology Summary

All application servers and the database server reside on the same internal network (192.168.24.x subnet). The database server (192.168.24.25) has NO public internet access. Outbound internet access from app servers is required for the following external services:

- Azure AD (login.microsoftonline.com:443) — for JWT/JWKS verification
- SAP BTP (*.cfapps.eu10-003.hana.ondemand.com:443) — for payslip SOAP API + OAuth2
- Posta Güvercini (www.postaguvercini.com:443, otpsms.postaguvercini.com:443) — for OTP SMS

- Gmail (smtp.gmail.com:587) — for email relay

Inbound connections to app servers come only from the internal corporate network (mobile devices on corporate Wi-Fi/VPN). The public-facing domain bizizapp.com serves the app download page and digital business card feature.